FinQuest — Gamified Financial Management System

Comprehensive Technical Documentation

Table of Contents

1. Executive Summary

FinQuest is a full-stack gamified personal financial management web application. Users track income and expenses, set budgets, create savings goals, and earn experience points (XP), achievements, and leaderboard rankings — turning financial discipline into an engaging game-like experience.

Key Value Propositions

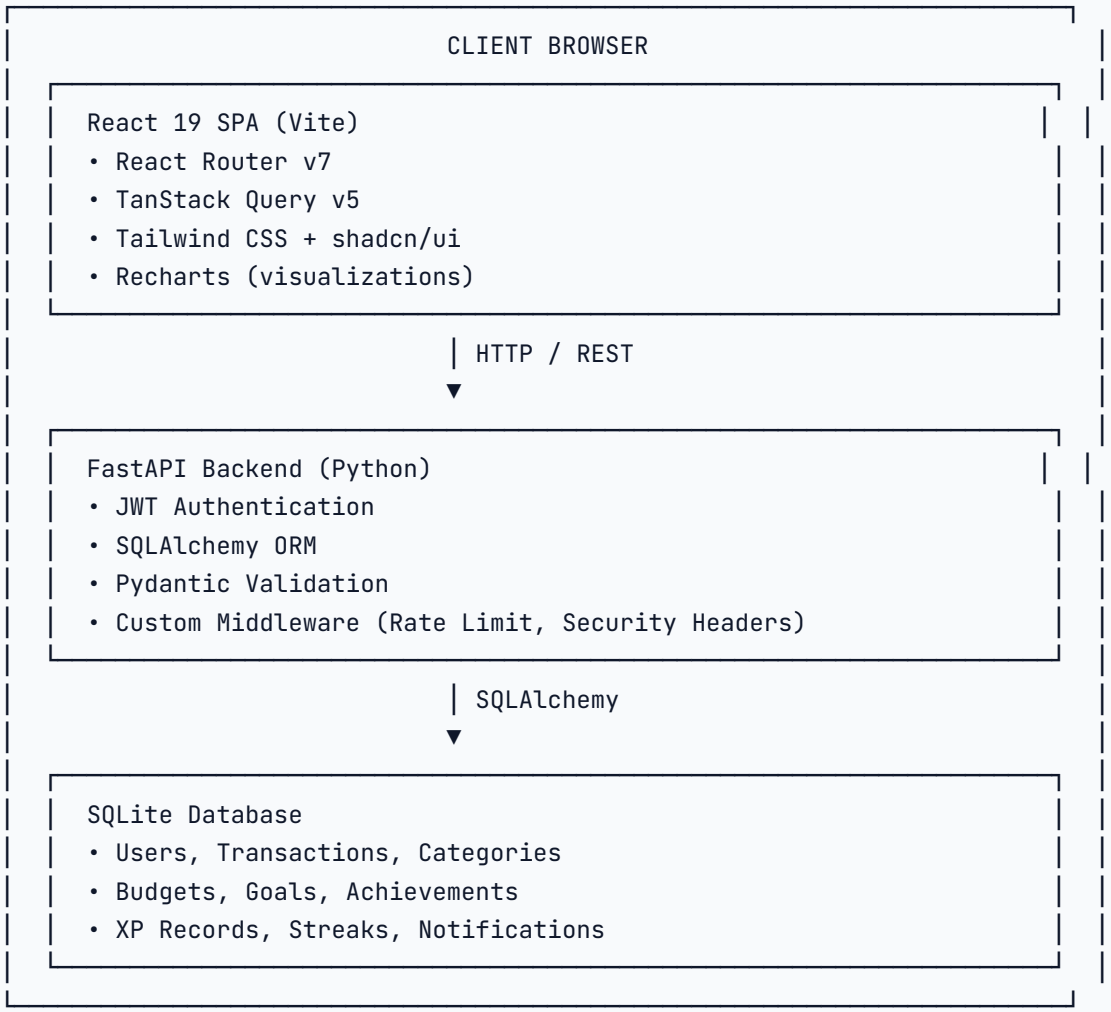
- **Gamification**: XP, levels, streaks, and achievements make budgeting fun
- **Comprehensive Tracking**: Income, expenses, budgets, and goals in one place
- **Visual Analytics**: Interactive charts and dashboards for spending insights
- **Data Portability**: Export/import transactions in CSV, JSON, and Excel formats
- **Secure by Design**: JWT authentication, bcrypt hashing, rate limiting, and security headers

Project Metadata

Attribute	Value
Project Name	FinQuest
Version	1.0.0
License	Private (Final Year Project)
Author	Israel
Date	May 2026

2. System Architecture Overview

High-Level Architecture



Architecture Style

- **Monolithic Full-Stack Application** with clear separation of concerns
- **Single-Page Application (SPA)** frontend served by the backend in production
- **RESTful API** communication between frontend and backend
- **Layered Backend**: Router → Service → Model → Database

3. Technology Stack

3.1 Backend Stack

Layer	Technology	Purpose
Framework	FastAPI 0.115	High-performance Python web framework with auto-generated OpenAPI docs
Language	Python 3.13	Backend programming language
ORM	SQLAlchemy 2.0	Database abstraction with declarative mapping
Database	SQLite	Lightweight file-based database (configurable to PostgreSQL/MySQL)
Validation	Pydantic v2	Request/response schema validation
Authentication	python-jose + passlib	JWT token generation and bcrypt password hashing
Rate Limiting	Custom Middleware	In-memory per-IP request throttling
Monitoring	Prometheus	Metrics collection via `prometheus-fastapi-instrumentator`
Logging	structlog	Structured JSON logging
Testing	pytest + pytest-asyncio	Unit and integration testing
Exports	openpyxl	Excel workbook generation

3.2 Frontend Stack

Layer	Technology	Purpose
Framework	React 19	UI library with concurrent features
Language	TypeScript 5.9	Type-safe JavaScript
Build Tool	Vite 7	Fast development server and optimized production builds
Routing	React Router v7	Client-side routing with nested layouts
Styling	Tailwind CSS 3.4	Utility-first CSS framework
UI Components	Radix UI + shadcn/ui	Accessible, composable UI primitives
State Management	TanStack Query v5	Server-state synchronization with caching
Charts	Recharts	Interactive data visualizations
Forms	React Hook Form + Zod	Performant form handling with schema validation
Icons	Lucide React	Modern icon library
Date Handling	date-fns	Date manipulation and formatting

3.3 Development Tools

Tool	Purpose
Git	Version control
ESLint	JavaScript/TypeScript linting
Prettier	Code formatting
Vitest	Frontend unit testing
pytest-cov	Backend test coverage

4. Design Patterns & Architectural Decisions

4.1 Backend Design Patterns

Application Factory Pattern

The FastAPI app is created via `create_app()` in `main.py`, allowing configuration-driven initialization. This enables easy testing with different configurations and supports lifespan events for database seeding.

```
def create_app() → FastAPI:
    app = FastAPI(lifespan=lifespan, ...)
    # Register middleware, routers, exception handlers
    return app
```

Dependency Injection

FastAPI's `Depends` is used extensively for:
- **Database Sessions**: `get_db()` yields SQLAlchemy sessions with automatic cleanup
- **Authentication**: `get_current_active_user()` injects the authenticated user into endpoints
- **Authorization**: Role-based access control via dependency chains

Repository Pattern (via Services)

Business logic is abstracted into service modules:
- `gamification_service.py` — XP/level/achievement/streak logic
- `export_service.py` — Data export formatting (CSV, JSON, Excel)
- `notification_service.py` — Notification generation and delivery

Generic API Response Wrapper

All endpoints return a standardized `ApiResponse[T]` structure:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2026-05-08T20:00:00",
    "request_id": "uuid",
    "pagination": { "page": 1, "limit": 20, "total": 100, "pages": 5 }
  }
}
```

This ensures consistent client-side error handling and response parsing.

Seed-on-Startup

Default data (10 categories, 14 achievements) is automatically seeded during application startup via the `lifespan` context manager, ensuring the app is immediately usable.

4.2 Frontend Design Patterns

Container/Presentational Pattern

Pages (containers) handle data fetching and business logic, while UI components (presentational) handle rendering. Example: `Dashboard.tsx` fetches analytics data and passes it to Recharts components.

Custom Hooks for Reusable Logic

- `useAuth()` — Authentication state, logout handling, and redirect logic
- `useTheme()` — Theme preference management with system preference detection

API Layer Abstraction

Two-tier API architecture:
1. **Low-level client** (`api/client.ts`) — Fetch wrapper with token injection and error handling
2. **High-level services** (`services/api.ts`) — Typed API modules for each domain (auth, transactions, budgets, etc.)

React Context for Global UI State

- `ThemeContext` — Dark/light/system theme with localStorage persistence and CSS variable switching
- `GamificationContext` — XP float animations, achievement toasts, and level-up modals

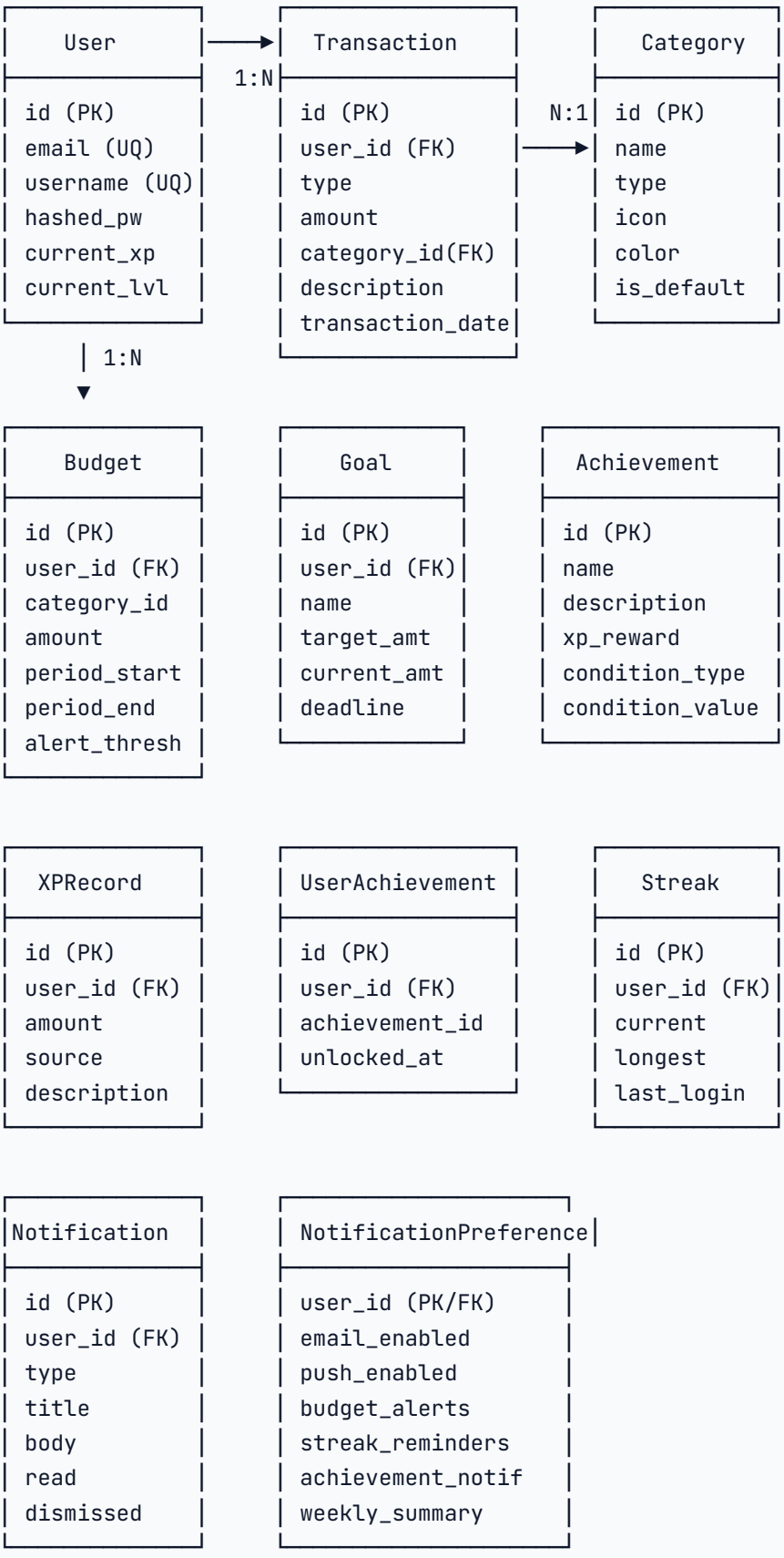
Layout-Based Routing

Authenticated pages share a common `Layout` component with sidebar navigation, gamification widgets, and achievement notifications. React Router's `<Outlet>` renders the matched child route.

---

5. Database Design

5.1 Entity Relationship Diagram


User
id (PK)
email (UQ)
username (UQ)
hashed_pw
current_xp
current_lvl
1:N
Transaction
id (PK)
user_id (FK)
type
amount
category_id(FK)
description
transaction_date
N:1
Category
id (PK)
name
type
icon
color
is_default
1:N
Budget
id (PK)
user_id (FK)
category_id
amount
period_start
period_end
alert_thresh
Goal
id (PK)
user_id (FK)
name
target_amt
current_amt
deadline
Achievement
id (PK)
name
description
xp_reward
condition_type
condition_value
XPRecord
id (PK)
user_id (FK)
amount
source
description
UserAchievement
id (PK)
user_id (FK)
achievement_id
unlocked_at
Streak
id (PK)
user_id (FK)
current
longest
last_login
Notification
id (PK)
user_id (FK)
type
title
body
read
dismissed
NotificationPreference
user_id (PK/FK)
email_enabled
push_enabled
budget_alerts
streak_reminders
achievement_notif
weekly_summary

5.2 Key Design Decisions

- **Single-table inheritance avoided**: Each entity has its own table with explicit foreign keys
- **Soft deletion not implemented**: Hard deletes for simplicity; could be added with `deleted_at` timestamps
- **Currency stored as string**: `currency` field on transactions with `"USD"` default; multi-currency support planned
- **Amounts stored as `Decimal`**: Prevents floating-point precision issues in financial calculations
- **User gamification fields denormalized**: `current_xp`, `current_level`, `total_xp_earned` stored directly on `User` for fast leaderboard queries

---

6. Backend Architecture

6.1 Project Structure

```
backend/
├── app/
│   ├── __init__.py
│   ├── main.py              # App factory, lifespan, router registration
│   ├── config.py            # Pydantic Settings (env vars)
│   ├── database.py          # SQLAlchemy engine, session, base
│   ├── dependencies.py      # Shared dependencies
│   ├── exceptions.py        # Custom exception hierarchy
│   ├── constants.py         # Default categories & achievement definitions
│   ├── middleware.py        # RateLimit, SecurityHeaders middleware
│   ├── auth/
│   │   ├── security.py      # bcrypt, JWT encode/decode
│   │   └── dependencies.py  # get_current_active_user
│   ├── models/              # SQLAlchemy ORM models
│   ├── routers/             # FastAPI route handlers
│   ├── schemas/             # Pydantic request/response models
│   └── services/            # Business logic
├── requirements.txt
└── seed_test_data.py        # Test data generator
```

6.2 Middleware Stack

Middleware is applied in this order (outermost first):

1. **CORS Middleware** — Cross-origin request handling for frontend development
2. **RateLimitMiddleware** — Per-IP request throttling (100 req/60s general, 5 req/15min auth)

3. **SecurityHeadersMiddleware** — X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy
4. **Request Metadata Middleware** — Injects `X-Request-ID` and `X-Response-Time-Ms` headers

6.3 Exception Handling

A global exception handler catches unhandled errors and returns sanitized responses:

```
{
  "success": false,
  "error": {
    "code": "INTERNAL_ERROR",
    "message": "An unexpected error occurred",
    "details": [],
    "timestamp": "2026-05-08T20:00:00Z",
    "request_id": "uuid"
  }
}
```

6.4 Configuration Management

All configuration is managed via Pydantic Settings with environment variable fallback:

Variable	Default	Description
`SECRET_KEY`	Auto-generated	JWT signing key
`ACCESS_TOKEN_EXPIRE_MINUTES`	30	Access token lifetime
`REFRESH_TOKEN_EXPIRE_DAYS`	7	Refresh token lifetime
`DATABASE_URL`	`sqlite:///./finquest.db`	Database connection string
`CORS_ORIGINS`	`http://localhost:5173,http://localhost:3000`	Allowed CORS origins
`RATE_LIMIT_REQUESTS`	100	Max requests per window
`RATE_LIMIT_WINDOW`	60	Rate limit window in seconds
`SECURE_COOKIES`	`False`	Cookie secure flag
`SAME_SITE`	`Lax`	Cookie SameSite attribute

7. Frontend Architecture

7.1 Project Structure

```
src/
├── main.tsx                # Entry point (React, Router, QueryClient, Providers)
├── App.tsx                 # Route definitions
├── index.css               # Global styles, CSS variables, animations
├── api/
│   └── client.ts           # Low-level fetch wrapper
├── components/
│   ├── Layout.tsx          # App shell (sidebar, gamification widget, toasts)
│   ├── AuthLayout.tsx      # Authentication page layout (unused)
│   └── ui/                 # shadcn/ui components (40+ primitives)
├── contexts/
│   ├── ThemeContext.tsx    # Dark/light/system theme management
│   └── GamificationContext.tsx # XP toasts, achievement notifications
├── hooks/
│   ├── useAuth.ts          # Authentication state hook
│   └── use-mobile.ts       # Mobile viewport detection
├── pages/
│   ├── Landing.tsx         # Marketing landing page
│   ├── Login.tsx           # Sign-in form
│   ├── Register.tsx        # Account creation form
│   ├── Dashboard.tsx       # Main dashboard with charts
│   ├── TransactionsPage.tsx # Transaction CRUD + filters
│   ├── BudgetsPage.tsx     # Budget management
│   ├── GoalsPage.tsx       # Savings goals + contributions
│   ├── AchievementsPage.tsx # Achievement gallery
│   ├── LeaderboardPage.tsx  # XP rankings
│   ├── Profile.tsx         # User settings, exports, notifications
│   └── NotFound.tsx        # 404 page
├── services/
│   └── api.ts              # Typed API service modules
├── const.ts                # Application constants
└── lib/
    └── utils.ts            # Utility functions (cn helper)
```

7.2 Theming System

The app uses CSS custom properties for theming with two approaches:

1. **App-specific variables** (`--bg-page` , `--text-primary` , etc.) defined in `index.css`
2. **shadcn/ui variables** (`--background` , `--primary` , etc.) for component consistency

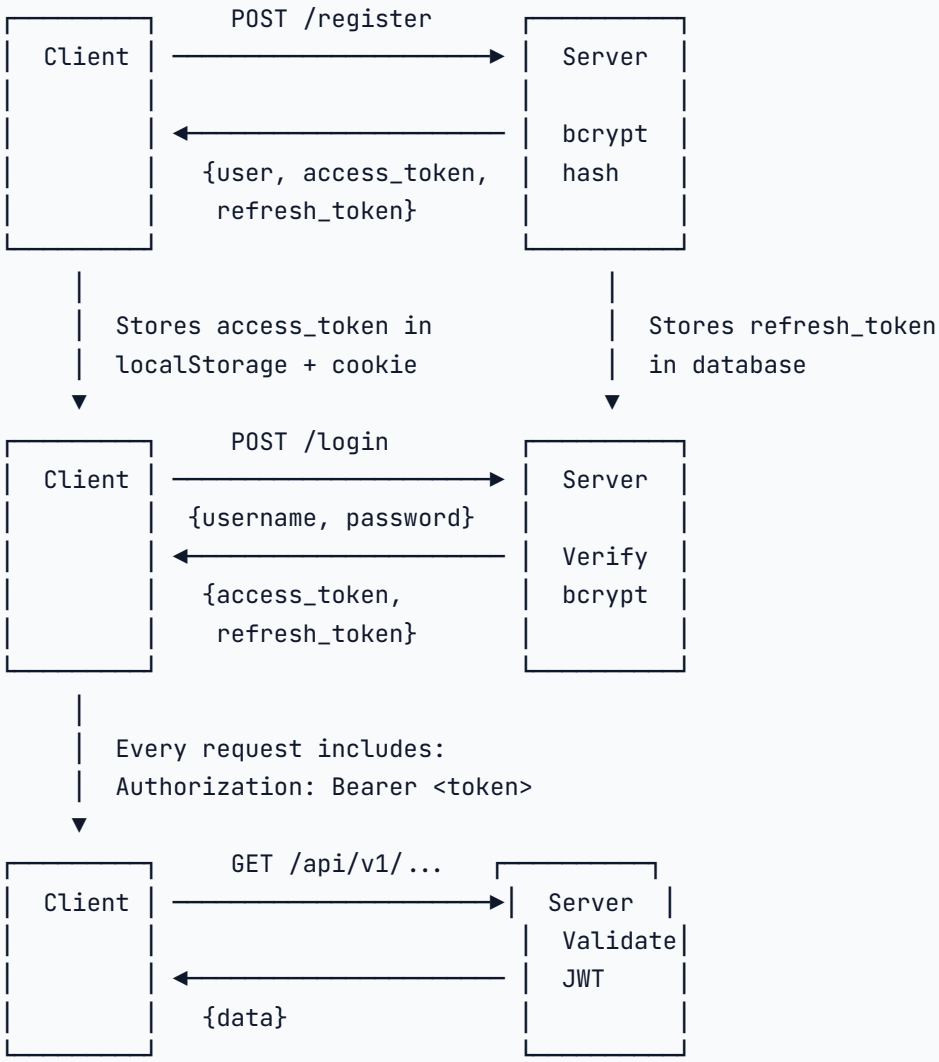
Theme switching works by:
- Setting `data-theme="light"` or `class="dark"` on the `<html>` element
- CSS selectors `:root[data-theme="light"]` and `.dark` override the variable values
- All components use CSS variables, ensuring instant theme switching without re-renders

7.3 State Management Strategy

State Type	Technology	Example
Server State	TanStack Query	Dashboard analytics, transaction lists, user profile
Global UI State	React Context	Theme preference, gamification toasts
Local Form State	useState	Login form inputs, filter selections
Auth Token	localStorage	JWT access token for API requests

8. Authentication & Security

8.1 Authentication Flow



8.2 Security Measures

Measure	Implementation	Details
Password Hashing	passlib + bcrypt	12 rounds of bcrypt
Access Tokens	JWT (HS256)	30-minute expiry
Refresh Tokens	JWT + database storage	7-day expiry, rotation on use
Token Delivery	Dual: JSON body + HttpOnly cookie	Cookie for SPA security, JSON for mobile
Rate Limiting	Custom middleware	5 req/15min for auth, 100 req/60s general
Security Headers	Custom middleware	X-Frame-Options, X-XSS-Protection, etc.
CORS	FastAPI CORSMiddleware	Whitelist-based origin filtering
SQL Injection Prevention	SQLAlchemy ORM	Parameterized queries only
XSS Prevention	React auto-escape	JSX automatically escapes output
Input Validation	Pydantic schemas	Strict type checking on all inputs

9. Gamification Engine

9.1 XP System

Action	XP Reward	Trigger
Add transaction	10 XP	`POST /transactions`
Daily login	20 XP	`POST /gamification/process-daily`
7-day streak	+50 XP bonus	Streak milestone
30-day streak	+200 XP bonus	Streak milestone
100-day streak	+1000 XP bonus	Streak milestone
Budget met	25 XP	Budget period completion
Goal 25%	+50 XP	Contribution milestone
Goal 50%	+100 XP	Contribution milestone
Goal 75%	+200 XP	Contribution milestone
Goal complete	+500 XP	Final contribution

9.2 Leveling Formula

```
Level = floor(Total XP Earned / 100) + 1
XP to Next Level = Level * 100
XP Progress = Current XP % 100
```

Example: A user with 730 XP is Level 8 (730/100 = 7.3, floor = 7, +1 = 8) with 30% progress to Level 9.

9.3 Streak System

- **Daily Login**: Users must log in on consecutive calendar days
- **Streak Tracking**: `Streak` model stores `current_streak`, `longest_streak`, and `last_login_date`
- **Milestone Bonuses**: Additional XP awarded at 7, 30, and 100-day streaks
- **Reset Logic**: Missing a calendar day resets `current_streak` to 1

9.4 Achievement System

14 pre-defined achievements seeded on startup:

Achievement	Condition	XP Reward
First Steps	Add first transaction	50 XP
Budget Master	Stay under budget for 3 periods	100 XP
Goal Setter	Create first goal	25 XP
Saver	Save $1000 total	150 XP
Big Saver	Save $5000 total	300 XP
Week Warrior	7-day login streak	75 XP
Month Master	30-day login streak	200 XP
Centurion	100-day login streak	500 XP
Diversified	Use 5 different categories	100 XP
Analyst	View analytics 10 times	50 XP
Export Pro	Export data 3 times	75 XP
Level 10	Reach level 10	250 XP
Level 25	Reach level 25	500 XP
Level 50	Reach level 50	1000 XP

10. API Specification

10.1 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	`/api/v1/auth/register`	Create new account	No
POST	`/api/v1/auth/login`	Authenticate and get tokens	No
POST	`/api/v1/auth/refresh`	Rotate refresh token	No
POST	`/api/v1/auth/logout`	Revoke refresh token	Yes
GET	`/api/v1/auth/me`	Get current user profile	Yes

10.2 Transaction Endpoints

Method	Endpoint	Description
GET	`/api/v1/transactions`	List transactions (paginated, filterable)
POST	`/api/v1/transactions`	Create new transaction
GET	`/api/v1/transactions/{id}`	Get transaction by ID
PUT	`/api/v1/transactions/{id}`	Update transaction
DELETE	`/api/v1/transactions/{id}`	Delete transaction
GET	`/api/v1/transactions/stats/summary`	Income/expense summary

10.3 Analytics Endpoints

Method	Endpoint	Description
GET	`/api/v1/analytics/dashboard`	Full dashboard data
GET	`/api/v1/analytics/spending-by-category`	Category breakdown with percentages
GET	`/api/v1/analytics/monthly-trend`	Income vs expense over N months

10.4 Gamification Endpoints

Method	Endpoint	Description
GET	`/api/v1/gamification/progress`	Current XP, level, streak
GET	`/api/v1/gamification/achievements`	Unlocked and locked achievements
GET	`/api/v1/gamification/dashboard`	Combined progress + achievements
POST	`/api/v1/gamification/process-daily`	Process daily login streak
GET	`/api/v1/gamification/leaderboard`	Top 50 users by XP

10.5 Export/Import Endpoints

Method	Endpoint	Description
GET	`/api/v1/export/transactions.csv`	CSV export with date range
GET	`/api/v1/export/full.json`	Full user data JSON export
POST	`/api/v1/export/csv`	Import transactions from CSV
POST	`/api/v1/export/json`	Import data from JSON

11. Feature Breakdown

11.1 Core Financial Features

Feature	Description	Status
Transaction Management	CRUD for income/expense with categories, dates, descriptions	✅ Complete
Category System	10 default categories + custom user categories	✅ Complete
Budget Tracking	Create budgets with amounts, date ranges, alert thresholds	✅ Complete
Goal Setting	Savings goals with target amounts, deadlines, contributions	✅ Complete
Analytics Dashboard	Monthly trends, category breakdown, summary statistics	✅ Complete
Data Export	CSV, JSON, Excel export formats	✅ Complete
Data Import	CSV and JSON import with validation	✅ Complete

11.2 Gamification Features

Feature	Description	Status
XP System	Earn XP for transactions, logins, streaks, goals	✅ Complete
Leveling	Automatic level calculation based on total XP	✅ Complete
Streaks	Daily login streak tracking with milestone bonuses	✅ Complete
Achievements	14 unlockable badges with progress tracking	✅ Complete
Leaderboard	Global XP rankings with top 50 display	✅ Complete

11.3 User Experience Features

Feature	Description	Status
Theme Switching	Light/dark/system mode with CSS variable theming	✅ Complete
Responsive Design	Mobile-friendly layout with collapsible sidebar	✅ Complete
Toast Notifications	Achievement unlocks and XP gain feedback	✅ Complete
Level-Up Modal	Full-screen celebration on level advancement	✅ Complete
Notification Preferences	Configurable alert types (budget, streak, achievements)	✅ Complete

---

12. Deployment & DevOps

12.1 Development Workflow

```
# Terminal 1 — Backend
cd app/backend
source venv/bin/activate
uvicorn app.main:app --host 127.0.0.1 --port 8001

# Terminal 2 — Frontend
cd app
npm run dev        # Vite dev server with API proxy
```

12.2 Production Deployment

Single-Server Deployment (Backend serves frontend):

```
cd app
npm run build                    # Build frontend to dist/public
cd backend
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8001
```

The backend automatically:
1. Serves static assets from `dist/public/assets/`
2. Returns `index.html` for all non-API routes (SPA fallback)
3. Handles API requests at `/api/v1/*`

12.3 Environment Variables for Production

```
# Required
SECRET_KEY="your-256-bit-secret-key-here"
DATABASE_URL="postgresql://user:pass@host:port/db"
ENVIRONMENT="production"

# Optional
CORS_ORIGINS="https://yourdomain.com"
SECURE_COOKIES=true
SAME_SITE="Strict"
```

---

13. Future Extensions

13.1 Planned Features

Feature	Description
Multi-Currency	Support for EUR, GBP, NGN with exchange rates
Recurring Transactions	Automatic monthly/weekly transaction generation
Social Features	Friend system, shared budgets, group challenges
AI Insights	Spending pattern analysis, budget recommendations
Mobile App	React Native or Flutter companion app
Push Notifications	Web push for budget alerts and streak reminders
Dark/Light Scheduling	Automatic theme switching based on time of day

13.2 Technical Debt

Item	Priority	Description
Database Migration	High	Add Alembic migrations for production schema changes
Test Coverage	High	Add comprehensive frontend and backend tests
Code Splitting	Medium	Reduce JS bundle size with dynamic imports
Type Safety	Medium	Eliminate `@ts-nocheck` comments
Dead Code Removal	Low	Remove unused tRPC/Hono dependencies

Appendix A: File Count Summary

Component	Files	Lines of Code
Backend	35+ Python files	~4,500 lines
Frontend	50+ TypeScript files	~6,000 lines
UI Components	40+ shadcn/ui components	~3,500 lines
Total	125+ files	~14,000 lines

Appendix B: Test Accounts

Username	Password	Purpose
`alice`	`password123`	Demo account with full data
`bob`	`password123`	Demo account with full data
`charlie`	`password123`	Demo account with full data
`demo`	`password123`	Demo account with full data

Document generated: May 2026
FinQuest v1.0.0 — Gamified Financial Management System